10.2.1 A step back: Introducing B-trees

Figure 10.2 shows an example of a B-tree, in particular a B+ tree; these data structures were meant to index unidimensional data, partitioning it into "*pages*"[3], providing efficient storage on disk and fast search (minimizing the number of pages loaded, and so the number of disk accesses).

Each node (both internal nodes or leaves) in a B-tree contains between `d-1` and `2*d-1` keys, where d is a fixed parameter for each tree, its branching factor[4]: the (minimum, in this case) number of children for each node. The only exception can be the root, which can possibly contain less than `d-1` keys. Keys are stored in an ordered list: this is fundamental to have a fast (logarithmic) search; in fact, each internal node with `m` keys, `k0, k1, ..., km-1`, `d-1 ≤ m ≤ 2*d*-1`, also has exactly `m+1` children, `C0, C1, ..., Cm-1, Cm` such that, `k < k0` for each key `k` in the subtree rooted in `C0`, `k0 ≤ k < k1` for each key `k` in the subtree rooted in `C1`, and so on...